Lab 2: Working with Self-Referential Data

Goals: The goals of this lab are to review data definitions and practice designing self-referential classes and data, then design methods for a variety of class herarchies.

Related files:
tester.jar javalib.jar

For this lab, there are no *starter files*. For each problem, start a new project and build the files from scratch.

2.1 Guided example

This section provides a guided walkthrough of translating Fundies 1-style data definitions into Java.

2.1.1 Initial scenario

Suppose we want to represent people and their modes of transportation, namely bicycles and cars. First, come up with some examples:

- Bob has a Diamondback bicycle.
- Ben has a Toyota which gets 30 miles per gallon.
- Becca has a Lamborghini which gets 17 miles per gallon.

One way to represent this information is as follows:

```
;; A MOT (ModeOfTransportation) is one of
;; -- Bicycle
;; -- Car

;; A Bicycle is a (make-bicycle String)
(define-struct bicycle (brand))
```

```
;; A Car is a (make-car String Number)
(define-struct car (make mpg))

;; A Person is a (make-person String MOT)
(define-struct person (name mot))

(define diamondback (make-bicycle "Diamondback"))
(define toyota (make-car "Toyota" 30))
(define lamborghini (make-car "Lamborghini" 17))

(define bob (make-person "Bob" diamondback))
(define ben (make-person "Ben" toyota))
(define becca (make-person "Becca" lamborghini))
```

Following Lecture 2, we can convert these data definitions into Java:

```
// Represents a mode of transportation
interface IMOT {}

// Represents a bicycle as a mode of transportation
class Bicycle implements IMOT {
  String brand;

  Bicycle(String brand) {
    this.brand = brand;
  }
}

// Represents a car as a mode of transportation
class Car implements IMOT {
  String make;
  int mpg; // represents the fuel efficiency in miles per gallon

  Car(String make, int mpg) {
    this.make = make;
    this.mpg = mpg;
  }
}

// Keeps track of how a person is transported
class Person {
  String name;
  IMOT mot;

  Person(String name, IMOT mot) {
```

```
    this.name = name;
    this.mot = mot;
  }
}
```

Then, our examples would become the following:

```
class ExamplesPerson {
  IMOT diamondback = new Bicycle("Diamondback");
  IMOT toyota = new Car("Toyota", 30);
  IMOT lamborghini = new Car("Lamborghini", 17);

  Person bob = new Person("Bob", diamondback);
  Person ben = new Person("Ben", toyota);
  Person becca = new Person("Becca", lamborghini);
}
```

Now that we have our data definitions and examples, let's try to write a method. Consider the following purpose statement:

```
// Does this person's mode of transportation meet the given fuel
// efficiency target (in miles per gallon)?
```

> ***Do Now!***
>
> How would you design a function like this in BSL?

Following the same design recipe steps as always, we need a purpose statement (given to us), a signature, some examples, and a template. Let's reorder these steps slightly, and first try to determine the template for this function, i.e. what data this method will work on. This will help us decide which interface or class should contain the method's implementation. The wording "this person's"suggests that this should be within the `Person` class, since it will be evaluating a person's method of transportation.

Next, we need to figure out the signature. A mode of transportation either meets a target fuel efficiency or it doesn't, so it would make sense for this method to return a `boolean`. Finally, we need to figure out this methods parameters (or if it even needs any). In this case, the method needs to be provided a target fuel efficiency, so one paramter `int mpg` should work. So our method header will look like the following:

```
// In the Person class:
boolean motMeetsFuelEfficiency(int mpg) { ... }
```

> ***Do Now!***

> Now that we have a signature and purpose, and figured out which class this method should be in, create some examples for how this method should behave.

Now it's time to try to implement the method, and to do that we need to work thorugh our available template. In doing so, we quickly realize that we do not have enough information. The only fields inside `Person` are `String name` and `IMOT mot`, neither of which can be directly compared to the given input.

> ***Do Now!***
>
> Why do we not want to simply write a `getMpg` method on `IMOT` and its implementing classes?

2.1.2 Solution: *delegation*

Because `Person` doesn't have enough information directly available, we need to find an available object of some other interface or class, and ask it to finish the work for us and return to us the answer. We call this approach *delegation*. In this example, we would likely want to delegate the work to the `IMOT` interface because the modes of transportation are the only ones that can truly tell us whether or not they are more fuel efficient than the given target fuel efficiency. Luckily, the result we want from this "wish list" method is the same as the return type for the method we are writing. Also, this new method would need the same input as our current method in order to evaluate fuel efficiency. Therefore, we could add this method header to `IMOT`:

```
// in IMOT
// returns true if this mode of transportation is at least
// as efficient as the given mpg, false otherwise
boolean isMoreFuelEfficientThan(int mpg);
```

Now, before we actually implement this method in `Bicycle` and `Car`, let's finish the method in `Person` *assuming* that our new method will work as intended. Since we now have this method on `IMOT`, we can simply invoke the method and return the result:

```
// in Person
boolean motMeetsFuelEfficiency(int mpg) {
  return this.mot.isMoreFuelEfficientThan(mpg);
}
```

Then, we can finish up the method in `Bicycle`:

```
// in Bicycle
// a bicycle does not consume fuel, so it will always be more fuel efficient
boolean isMoreFuelEfficientThan(int mpg) {
  return true;
}
```

and `Car`:

```
// in Car
// compare this car's fuel efficiency to the given fuel efficiency
boolean isMoreFuelEfficientThan(int mpg) {
  return this.mpg >= mpg;
}
```

Do Now!

Write tests on our examples for a target fuel efficiency of 15 mpg. What about 25 mpg?

Exercise

How would we do this if we wanted to compare to another mode of transportation's fuel efficiency instead of a given target fuel efficiency? More precisely, what if we wanted to change the method header in `Person` to the following:

```
boolean motIsMoreFuelEfficientThan(IMOT mot)
```

Where do you get stuck?

2.2 Methods for Structured Data

Here are some unfinished classes that represent pets and pet owners:

```
// to represent a pet owner
class Person {
    String name;
    IPet pet;
    int age;
```

```
    Person(String name, IPet pet, int age) {
        this.name = name;
        this.pet = pet;
        this.age = age;
    }
}

// to represent a pet
interface IPet { }

// to represent a pet cat
class Cat implements IPet {
    String name;
    String kind;
    boolean longhaired;

    Cat(String name, String kind, boolean longhaired) {
        this.name = name;
        this.kind = kind;
        this.longhaired = longhaired;
    }
}

// to represent a pet dog
class Dog implements IPet {
    String name;
    String kind;
    boolean male;

    Dog(String name, String kind, boolean male) {
        this.name = name;
        this.kind = kind;
        this.male = male;
    }
}
```

Follow the design recipe to design the following methods for these classes:

- Make examples of at least four pet owners - two for each of the pets, as you need to make sure you cover the two `boolean` values.
- Design a method with the following purpose and header:

```
// is this Person older than the given Person?
```

```
boolean isOlder(Person other)
```

- Design a method `sameNamePet` that has the following purpose statement:

  ```
  // does the name of this person's pet match the given name?
  ```

  We've given you the purpose statement, so start with examples and the template. When you make the template for the class `Person`, you should notice that you do not know anything about the pet's name. You will therefore need a helper method, somewhere in the class hierarchy that represents `IPet`s, that will check whether this pet's name matches the given name.

  But to design such a method, you first need to define the purpose statements and the header in the interface `IPet`. Remember, Java requires that this method be declared `public` - though at this time, for our simple programs, this designation is irrelevant.

  > We will soon learn how we can avoid the extra work.

  You then need to define this method in every class that implements the interface.

- Some people do not have pets, and sometimes–sadly–the pets do perish. Add a new class `NoPet` to this class hierarchy so that we can represent `Person`s who do not have a pet.

  Make sure you add new examples!

- Design a method `perish` in the class `Person` that produces a person whose pet has perished.